

Nomes Diversos - As palavras ocultas podem ser encontradas
no sentidos: Horizontal, Vertical ou Diagonal

E	R	W	A	I	Y	D	P	I	M	X	A	P	I	R	E	H	L	U	M	ALFINETE
A	H	S	C	L	E	Q	T	Y	E	E	C	I	C	N	J	F	O	P	L	BANHEIRO
X	L	Q	Z	P	F	C	D	H	C	C	M	L	P	A	C	A	T	X	U	BRINCO
F	X	A	E	C	Q	I	S	I	C	E	F	O	Z	P	M	C	R	A	Q	BULE
O	S	R	S	B	O	N	N	N	T	N	D	S	H	G	S	A	A	H	A	CADEIRA
K	I	I	X	A	T	Z	I	E	I	S	P	A	R	Q	U	E	U	L	A	CAMA
M	Z	E	P	C	J	F	I	A	T	S	M	O	D	M	R	G	O	A	T	CIDADE
Y	I	D	N	R	L	B	A	N	H	E	I	R	O	I	H	V	O	O	S	COLHER
L	V	A	P	X	E	O	G	S	H	B	L	D	E	V	C	L	A	T	A	COPO
S	D	C	X	Z	C	S	E	B	B	A	L	G	O	S	O	X	P	A	P	COZINHA
H	S	L	G	G	N	R	E	H	F	J	U	J	A	R	R	O	A	A	M	FACA
F	I	A	G	K	I	V	R	N	W	M	K	E	M	J	D	V	R	M	G	FESTA
S	E	W	B	P	P	E	T	D	T	V	G	R	Z	E	A	B	E	B	N	HOMEM
Y	T	S	R	O	H	A	N	I	N	E	M	Q	O	S	U	R	D	E	P	JARRO
Y	E	J	T	L	N	I	F	T	O	E	R	R	S	I	Q	Q	E	N	L	MENINA
Y	P	H	O	A	Z	E	J	N	N	J	I	O	W	O	T	A	R	P	T	MENINO
Z	A	C	P	E	Z	D	T	I	Z	O	U	A	P	P	A	N	E	L	A	MULHER
B	T	R	H	L	D	M	N	E	B	R	I	N	C	O	D	I	S	P	L	PANELA
D	T	I	B	U	C	O	P	O	A	N	A	Z	Q	I	H	I	M	N	S	PAREDE
E	T	G	A	B	Y	L	X	K	M	H	Q	S	B	U	G	C	P	M	O	PARQUE
																				PASTA
																				PIA
																				PINCEL
																				PIRES
																				PRATO
																				PRESENTE
																				GUARDO

Palavras em uma Matriz

Felippe Carballo Leal

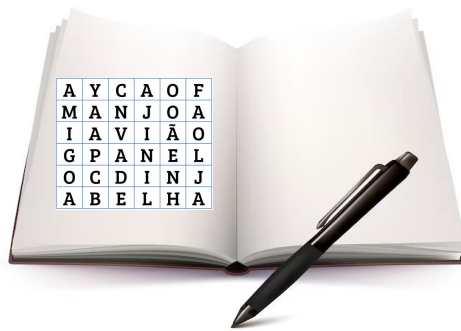
João Luiz Schiavini Filho

Repositório: [aed2-trabalho2-gp4-2025.2](#)



O problema

Dada uma matriz de caracteres e uma lista de palavras, **é possível encontrar** uma combinação de **letras consecutivas adjacentes, sem repetir**, que forme uma **palavra da lista**?



AVIÃO

ANJO

...

—
Qual é a melhor solução?

Backtracking

Por quê?

— Por quê **Backtracking**?

Se usássemos...

Programação Dinâmica

Essa solução não é eficaz pois utilizaria a matriz inteira, e repetiria combinações que já não funcionaram em uma primeira verificação.

Se usássemos...

Um Algoritmo Guloso

O algoritmo guloso não se arrepende de escolhas já feitas. Dada uma cadeia e um caminho que não consegue abranger completamente a cadeia, não será feita outra tentativa caso o algoritmo encontre este caminho.

Não existe solução com algoritmo guloso para esse problema.

O **Backtracking** não repete caminhos já visitados e desfaz decisões ruins

Se usássemos...

Programação Dinâmica

>>BÜ, %iBÖ
ÜÄä-'A=
z%GÜËsiÜ
DÄNxiSBÄ

Resposta (dinâmica):
ruídos de internet discada



Se usássemos...

Um Algoritmo Guloso

A	Y	C	A	O	F
M	A	N	Ã	O	A
I	A	V	I	J	O
G	P	A	N	E	L
O	C	D	I	N	J
A	B	E	L	H	A

Resposta (gulosa):
Encontramos! ← ?

Solucionando com Backtracking

- **Busca em Profundidade** (em inglês, DFS ou Depth-First Search)
 - Algoritmo de busca de nós em grafos
 - Explora todos os caminhos antes de voltar (backtrack)

O algoritmo foi implementado em python com os seguintes procedimentos:

- **Marcação** - A célula atual é marcada para não ser repetida.
- **Exploração** - Dada uma posição, a função é chamada recursivamente para as 8 adjacentes.
- **Backtracking** - Se o caminho não levar à palavra completa, a célula é desmarcada.
- **Parada** - Quando é encontrada a palavra, a busca é encerrada e retorna.



O Código

```
DIRECOES = [
    (0, 1), (0, -1), (1, 0), (-1, 0),
    (1, 1), (1, -1), (-1, 1), (-1, -1)
]

def busca_backtracking(
    matriz: List[List[str]], palavra: str, linha: int,
    coluna: int, indice: int ) -> bool:

    if indice == len(palavra):
        return True

    if (linha < 0 or linha >= len(matriz) or
        coluna < 0 or coluna >= len(matriz[0]) or
        matriz[linha][coluna] != palavra[indice] or
        matriz[linha][coluna] == '#'):
        return False

    char_original = matriz[linha][coluna]
    matriz[linha][coluna] = '#'

    encontrado = False
    for dr, dc in DIRECOES:
        if busca_backtracking(matriz, palavra, linha + dr,
            coluna + dc, indice + 1):
            encontrado = True
            break

    matriz[linha][coluna] = char_original

    return encontrado
```

Caso o índice a verificar seja 1 após a última letra (verificar $n+1$ ésima letra), retorne Verdadeiro (significa que já foi encontrada).

Caso a posição não exista (está fora do limite), a letra na matriz não corresponda, ou a letra na matriz já foi visitada, retorne Falso.

Marcação da letra visitada na matriz, dado que ela corresponde e não foi visitada ainda.

Nova exploração, nas 8 possíveis direções. O resultado é repassado na volta.

Desmarcação da letra na matriz e retorno.



O Código

```
def palavras_matriz(matriz: List[List[str]],
                    palavras: List[str]) -> Dict[str, Tuple[bool, float]]:

    resultados = {}

    for palavra in palavras:
        if not palavra:
            resultados[palavra] = False
            continue

        M = len(matriz)
        N = len(matriz[0]) if M > 0 else 0
        encontrado = False

        for i in range(M):
            for j in range(N):
                if matriz[i][j] == palavra[0]:
                    matriz_copia = [lin[:] for lin in matriz]
                    if busca_backtracking(matriz_copia,
                                         palavra, i, j, 0):
                        encontrado = True
                        break
            if encontrado:
                break

        resultados[palavra] = encontrado

    return resultados
```

Se a palavra está vazia, não fazer a procura dela, e marcar como “não encontrada”

Loop para buscar cadeias correspondentes na matriz. Usa a função `busca_backtracking`, apresentada no slide anterior.

Caso a palavra tenha sido encontrada, saia do loop...

...e armazene o resultado da busca.



Execução

```
Palavra: 'CANTO'  
Encontrada? Não  
Tempo de Execução: 0.017881 milissegundos  
-----
```

```
Palavra: 'LAGO'  
Encontrada? Não  
Tempo de Execução: 0.002146 milissegundos  
-----
```

```
Palavra: 'TERAPIA'  
Encontrada? Sim  
Tempo de Execução: 0.007391 milissegundos  
-----
```

```
Palavra: 'SIM'  
Encontrada? Sim  
Tempo de Execução: 0.006437 milissegundos  
-----
```

```
Palavra: 'AMOR'  
Encontrada? Sim  
Tempo de Execução: 0.003815 milissegundos  
-----
```

```
Palavra: 'CASA'  
Encontrada? Não  
Tempo de Execução: 0.004768 milissegundos  
-----
```

C	T	E	D	O
E	A	R	N	A
P	A	M	O	S
O	I	S	R	T

CANTO
LAGO
TERAPIA
SIM
AMOR
CASA



Execução

Palavra: 'CANTO'
Encontrada? Não
Tempo de Execução: 0.017881 milissegundos

Palavra: 'LAGO'
Encontrada? Não
Tempo de Execução: 0.002146 milissegundos

Palavra: 'TERAPIA'
Encontrada? Sim
Tempo de Execução: 0.007391 milissegundos

Palavra: 'SIM'
Encontrada? Sim
Tempo de Execução: 0.006437 milissegundos

Palavra: 'AMOR'
Encontrada? Sim
Tempo de Execução: 0.003815 milissegundos

Palavra: 'CASA'
Encontrada? Não
Tempo de Execução: 0.004768 milissegundos

C	T	E	D	O
E	A	R	N	A
P	A	M	O	S
O	I	S	R	T

CANTO
LAGO
TERAPIA
SIM
AMOR
CASA



Execução

```
Palavra: 'CANTO'  
Encontrada? Não  
Tempo de Execução: 0.017881 milissegundos  
-----
```

```
Palavra: 'LAGO'  
Encontrada? Não  
Tempo de Execução: 0.002146 milissegundos  
-----
```

```
Palavra: 'TERAPIA'  
Encontrada? Sim  
Tempo de Execução: 0.007391 milissegundos  
-----
```

```
Palavra: 'SIM'  
Encontrada? Sim  
Tempo de Execução: 0.006437 milissegundos  
-----
```

```
Palavra: 'AMOR'  
Encontrada? Sim  
Tempo de Execução: 0.003815 milissegundos  
-----
```

```
Palavra: 'CASA'  
Encontrada? Não  
Tempo de Execução: 0.004768 milissegundos  
-----
```

C	T	E	D	O
E	A	R	N	A
P	A	M	O	S
O	I	S	R	T

CANTO
LAGO
TERAPIA
SIM
AMOR
CASA



Execução

```
Palavra: 'CANTO'  
Encontrada? Não  
Tempo de Execução: 0.017881 milissegundos  
-----
```

```
Palavra: 'LAGO'  
Encontrada? Não  
Tempo de Execução: 0.002146 milissegundos  
-----
```

```
Palavra: 'TERAPIA'  
Encontrada? Sim  
Tempo de Execução: 0.007391 milissegundos  
-----
```

```
Palavra: 'SIM'  
Encontrada? Sim  
Tempo de Execução: 0.006437 milissegundos  
-----
```

```
Palavra: 'AMOR'  
Encontrada? Sim  
Tempo de Execução: 0.003815 milissegundos  
-----
```

```
Palavra: 'CASA'  
Encontrada? Não  
Tempo de Execução: 0.004768 milissegundos  
-----
```

C	T	E	D	O
E	A	R	N	A
P	A	M	O	S
O	I	S	R	T

CANTO
LAGO
TERAPIA
SIM
AMOR
CASA

Complexidade do algoritmo

$$T(M, N, L) = M \cdot N \cdot O(8^L)$$

- **Complexidade de Tempo (Pior Caso):** $O(M \cdot N \cdot 8^L)$
 - $M \cdot N$ representam a busca em cada célula da matriz.
 - 8^L representa os 8 caminhos possíveis para cada célula analisada.
- **Complexidade de Espaço (Pior Caso):** $O(L + M \cdot N)$
 - $O(L)$ para o máximo de chamadas recursivas empilhadas.
 - $O(M \cdot N)$ devido ao uso inerente da matriz.

Onde M e N são as dimensões da matriz, e L é o comprimento da palavra a ser encontrada.



Curiosidade

Na *programação dinâmica*, o problema é pior que exponencial.

Se tentássemos um *algoritmo guloso* para fazer algo parecido, a complexidade seria algo em torno de $O(M \cdot N \cdot L)$ para tempo, e $O(1)$ para espaço.

Fim!



Alunos

João Luiz Schiavini Filho

Felipe Carballo Leal

Repositório (QR-Code)

<https://github.com/felipecarballo/aed2-trabalho2-gp4-2025.2>



Aviso: Não estamos sendo patrocinados pela Editora Coquetel